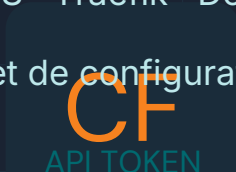


# Cloudflare API Token

SOPS · Traefik · Docker

Guide complet de configuration sécurisée



romain-ecarnot.com · Avril 2026

Romain Ecarnot — Saint-Sébastien-sur-Loire



# Table des matières

---

|                                                          |    |
|----------------------------------------------------------|----|
| 1 Prérequis et vue d'ensemble .....                      | 3  |
| 1.1 Architecture de la solution .....                    | 3  |
| 1.2 Outils nécessaires .....                             | 3  |
| 2 Créer le Token API Cloudflare .....                    | 4  |
| 2.1 Accès au tableau de bord Cloudflare .....            | 4  |
| 2.2 Création du token avec permissions restreintes ..... | 4  |
| 2.3 Permissions Zone:DNS:Edit et Zone:Zone:Read .....    | 5  |
| 2.4 Restriction au domaine romain-ecarnot.com .....      | 5  |
| 2.5 Vérification du token .....                          | 6  |
| 3 Chiffrement avec SOPS .....                            | 7  |
| 3.1 Installation de SOPS et age .....                    | 7  |
| 3.2 Configuration avec age (recommandé) .....            | 7  |
| 3.3 Configuration avec PGP/GPG (alternative) .....       | 8  |
| 3.4 Chiffrer le token Cloudflare .....                   | 9  |
| 3.5 Intégration dans le workflow .....                   | 9  |
| 4 Configuration Docker Traefik .....                     | 10 |
| 4.1 Structure du projet .....                            | 10 |
| 4.2 Configuration statique traefik.yml .....             | 10 |
| 4.3 Docker Compose complet .....                         | 11 |
| 4.4 Certificats ACME DNS Challenge .....                 | 12 |
| 4.5 Services avec TLS automatique .....                  | 13 |
| 5 Vérifications et erreurs courantes .....               | 14 |
| 5.1 Tests de validation .....                            | 14 |
| 5.2 Erreurs fréquentes et solutions .....                | 14 |
| 5.3 Commandes de diagnostic .....                        | 15 |

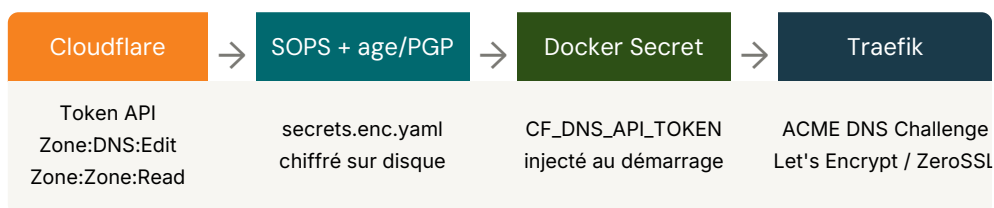
## 1

## Prérequis et vue d'ensemble

Outils, accès et architecture de la solution

### Architecture de la solution

Ce guide configure un pipeline de sécurité en trois couches : le token API Cloudflare est créé avec des permissions minimales (principe du moindre privilège), chiffré au repos avec SOPS, puis injecté de manière sécurisée dans Traefik pour automatiser l'émission de certificats TLS via DNS Challenge.



### Outils nécessaires

| Outil              | Version | Installation          | Rôle                                   |
|--------------------|---------|-----------------------|----------------------------------------|
| Cloudflare account | —       | dash.cloudflare.com   | Gestionnaire DNS du domaine            |
| Docker + Compose   | ≥ 26    | docker.com/get-docker | Orchestration des services             |
| Traefik            | v3.x    | image traefik:v3      | Reverse proxy + TLS automatique        |
| SOPS               | ≥ 3.9   | brew install sops     | Chiffrement des secrets                |
| age                | ≥ 1.1   | brew install age      | Algorithme de chiffrement (recommandé) |
| GPG (optionnel)    | ≥ 2.4   | brew install gnupg    | Alternative PGP à age                  |

2

## Créer le Token API Cloudflare

Permissions minimales · Restriction de domaine

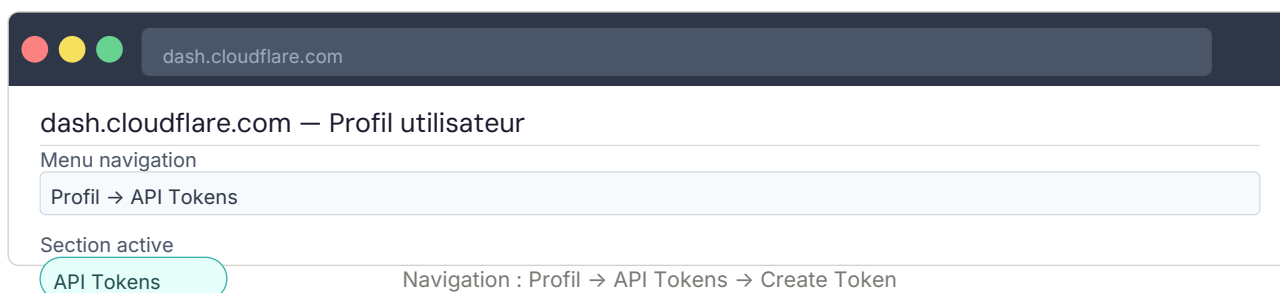
### 2.1 — Accès au tableau de bord Cloudflare

Connectez-vous à [dash.cloudflare.com](https://dash.cloudflare.com), puis naviguez vers votre profil utilisateur.

1

#### Ouvrir les paramètres du profil

Cliquer sur l'icône de profil en haut à droite



### 2.2 — Création du token avec permissions restreintes

Cloudflare propose des modèles de tokens prédéfinis. Pour Traefik, sélectionnez « Edit zone DNS » comme point de départ, puis affinez les permissions.

2

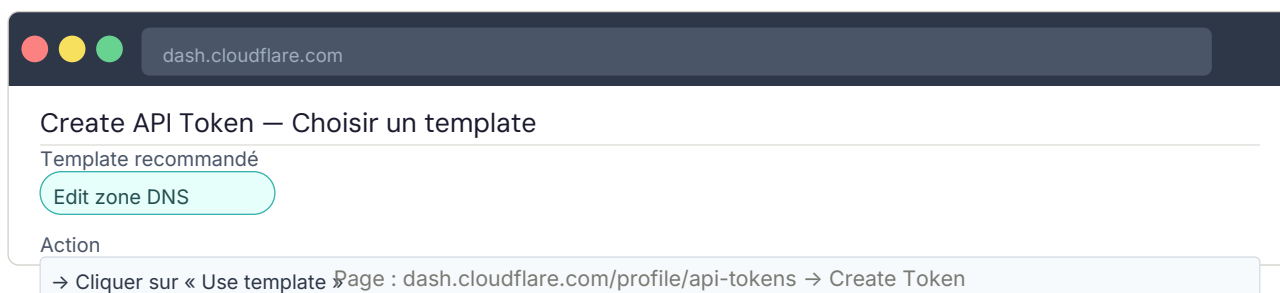
#### Cliquer sur « Create Token »

Bouton bleu en haut à droite de la page API Tokens

3

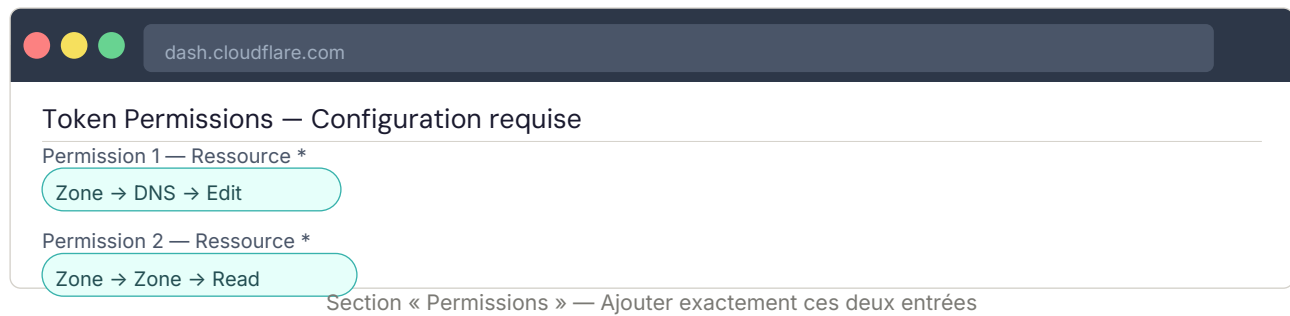
#### Sélectionner le template « Edit zone DNS »

Panneau « API token templates » → Use template



### 2.3 — Configurer les permissions Zone

Principe du moindre privilège : Le token doit avoir uniquement les permissions nécessaires pour le DNS Challenge. Aucune permission sur les règles de pare-feu, workers, ou autres ressources.



dash.cloudflare.com

### Token Permissions — Configuration requise

Permission 1 — Ressource \*

Zone → DNS → Edit

Permission 2 — Ressource \*

Zone → Zone → Read

Section « Permissions » — Ajouter exactement ces deux entrées

| Ressource | Catégorie | Niveau d'accès | Utilité                                                         |
|-----------|-----------|----------------|-----------------------------------------------------------------|
| Zone      | DNS       | Edit           | Créer / modifier les enregistrements TXT pour le challenge ACME |
| Zone      | Zone      | Read           | Lire les informations de la zone (ID de zone, nom de domaine)   |

## 2.4 — Restriction au domaine romain-ecarnot.com

La section « Zone Resources » permet de limiter le token à un domaine spécifique. C'est une mesure de sécurité critique — si le token est compromis, il ne pourra agir que sur romain-ecarnot.com.

4

### Configurer Zone Resources

Section « Zone Resources » dans le formulaire de création



dash.cloudflare.com

### Zone Resources — Restriction du domaine

Sélecteur de portée

Specific zone

Domaine cible \*

romain-ecarnot.com

Important : choisir « Specific zone » et non « All zones »

⚠ Ne jamais utiliser « All zones » — cela donnerait accès à tous les domaines de votre compte Cloudflare. Toujours restreindre à romain-ecarnot.com uniquement.

## 2.5 — Récupérer et vérifier le token

5

### Valider et copier le token

Cliquer sur « Continue to summary » puis « Create Token »

Le token n'est affiché qu'une seule fois. Copiez-le immédiatement dans un gestionnaire de mots de passe ou chiffrez-le avec SOPS (section 3). Si perdu, il faut en créer un nouveau.

Test de validation du token via curl :

```
terminal – Vérification du token

# Remplacer YOUR_TOKEN par le token récupéré
export CF_DNS_API_TOKEN="YOUR_TOKEN"

# Vérifier que le token est valide
curl -s -X GET "https://api.cloudflare.com/client/v4/user/tokens/verify" \
  -H "Authorization: Bearer ${CF_DNS_API_TOKEN}" \
  -H "Content-Type: application/json" | jq .

# Réponse attendue :
{ "result": { "status": "active" }, "success": true }

# Vérifier l'accès à la zone romain-ecarnot.com
curl -s "https://api.cloudflare.com/client/v4/zones?name=romain-ecarnot.com" \
  -H "Authorization: Bearer ${CF_DNS_API_TOKEN}" | jq .result[0].id
```

## 3

## Chiffrement avec SOPS

age (recommandé) · PGP/GPG (alternative)

SOPS (Secrets OPERATIONs) de Mozilla permet de chiffrer des fichiers de configuration tout en conservant leur structure (YAML, JSON, ENV). Les clés restent visibles, seules les valeurs sont chiffrées — idéal pour Git.

### 3.1 — Installation de SOPS et age

```
terminal — Installation

# macOS (Homebrew)
brew install sops age

# Vérification des versions
sops --version      # ex: sops 3.9.4
age --version       # ex: v1.1.1

# Linux (apt)
curl -LO https://github.com/getsops/sops/releases/latest/download/sops-v3.9.4.linux.amd64
sudo install sops-v3.9.4.linux.amd64 /usr/local/bin/sops
sudo apt install age # ou télécharger depuis releases GitHub
```

### 3.2 — Configuration avec age (recommandé)

age est la méthode recommandée : plus simple que PGP, pas de gpg-agent, clés légères stockées en fichiers texte.

```
terminal — Génération des clés age

# 1. Générer une paire de clés age
mkdir -p ~/.config/sops/age
age-keygen -o ~/.config/sops/age/keys.txt

# Sortie (exemple) :
# Public key: age1ql3z7hgy54pw3hyww5aayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p

# 2. Afficher la clé publique (à copier pour .sops.yaml)
grep 'public key' ~/.config/sops/age/keys.txt

# 3. Configurer la variable d'environnement
echo 'export SOPS_AGE_KEY_FILE="$HOME/.config/sops/age/keys.txt"' >> ~/.zshrc
source ~/.zshrc
```

Créer le fichier de configuration SOPS `.sops.yaml` :

Placez ce fichier à la racine de votre projet Docker/Traefik :

```
.sops.yaml

# .sops.yaml – racine du projet
creation_rules:
  - path_regex: secrets\.enc\.yaml$
    age:
      - age1ql3z7hjy54pw3hyww5ayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p
      # Ajouter d'autres clés publiques pour un accès multi-développeur :
      # - age1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
  - path_regex: \.env\.enc$
    age:
      - age1ql3z7hjy54pw3hyww5ayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p
```

### 3.3 – Configuration avec PGP/GPG (alternative)

PGP/GPG est une alternative robuste si vous avez déjà une infrastructure GPG en place (YubiKey, Kleopatra, etc.). La configuration est plus complexe qu'age mais offre des fonctionnalités d'entreprise (GPG agents, smart cards).

```
terminal – Configuration GPG

# 1. Générer une paire de clés GPG (si inexistante)
gpg --full-generate-key
# Choisir : RSA 4096 bits, sans expiration (ou avec), entrer nom/email

# 2. Récupérer le fingerprint de la clé
gpg --list-secret-keys --keyid-format LONG
# ex: sec  rsa4096/ABC123DEF456GHIJ 2024-01-01 ...
# Le fingerprint est : ABC123DEF456GHIJ

# 3. Configurer .sops.yaml avec GPG
# creation_rules:
#   - path_regex: secrets\.enc\.yaml$
#     pgp:
#       - ABC123DEF456GHIJ1234567890ABCDEF12345678
```

#### Comparaison age vs PGP

| Critère           | age                          | PGP/GPG                        |
|-------------------|------------------------------|--------------------------------|
| Simplicité        | Très simple                  | Complexe (gpg-agent, keyrings) |
| Sécurité          | Moderne (ChaCha20+Poly1305)  | Éprouvée (RSA/Curve25519)      |
| Hardware keys     | Non natif                    | YubiKey, smart cards           |
| Multi-utilisateur | Via clés publiques multiples | Via keyring partagé            |
| Recommandation    | ✓ Projets personnels/équipes | ✓ Infrastructure entreprise    |

## 3.4 — Chiffrer le token Cloudflare avec SOPS

6

### Créer le fichier de secrets en clair

Ne jamais committer ce fichier dans Git !

```
terminal — Chiffrement du token

# Créer le fichier secrets.yaml (NON chiffré, temporaire)
cat > /tmp/secrets.yaml << 'EOF'
cloudflare:
  CF_DNS_API_TOKEN: "votre-token-cloudflare-ici"
  CF_ZONE_ID: "votre-zone-id-ici"
  CF_EMAIL: "romain@romain-ecarnot.com"
EOF

# Chiffrer avec SOPS (lit .sops.yaml automatiquement)
sops --encrypt /tmp/secrets.yaml > secrets.enc.yaml

# Supprimer le fichier en clair
rm /tmp/secrets.yaml

# Vérifier le contenu chiffré
cat secrets.enc.yaml
# Les valeurs sont chiffrées, les clés restent lisibles :
# cloudflare:
#   CF_DNS_API_TOKEN: ENC[AES256_GCM,data:xxxx...]
```

## 3.5 — Déchiffrement et intégration

```
terminal — Déchiffrement et injection

# Déchiffrer pour lire (ne crée pas de fichier en clair)
sops --decrypt secrets.enc.yaml

# Exporter une variable spécifique vers l'environnement
export CF_DNS_API_TOKEN=$(sops --decrypt --extract '["cloudflare"]["CF_DNS_API_TOKEN"]' secrets.enc.yaml)

# Générer un .env déchiffré pour Docker (temporaire)
sops --decrypt --output-type dotenv secrets.enc.yaml > /tmp/.env.traefik
docker compose --env-file /tmp/.env.traefik up -d
rm /tmp/.env.traefik # nettoyer après usage

# Script d'aide : scripts/start-traefik.sh
#!/bin/bash
set -e
export CF_DNS_API_TOKEN=$(sops -d --extract '["cloudflare"]["CF_DNS_API_TOKEN"]' secrets.enc.yaml)
docker compose up -d
```

▲ Ajouter secrets.yaml et .env dans .gitignore. Seul secrets.enc.yaml et .sops.yaml doivent être commités.

## 4

## Configuration Docker Traefik

DNS Challenge Cloudflare · ACME · TLS automatique

### 4.1 — Structure du projet

```
Structure du projet

traefik/
├── docker-compose.yml    # Orchestration principale
├── traefik.yml           # Configuration statique Traefik
├── .sops.yaml            # Règles de chiffrement SOPS
├── secrets.enc.yaml      # Secrets chiffrés (versionnable)
├── scripts/
│   └── start.sh          # Script de démarrage avec déchiffrement
├── config/
│   ├── dynamic/          # Configurations dynamiques
│   │   ├── middlewares.yml # Middlewares (redirect HTTPS, etc.)
│   │   └── tls.yml        # Options TLS globales
│   └── data/
│       └── acme.json       # Certificats Let's Encrypt (chmod 600)
```

### 4.2 — Configuration statique traefik.yml

Le fichier traefik.yml définit les resolvers ACME et les entrypoints :

```
traefik.yml

# traefik.yml - Configuration statique
global:
  checkNewVersion: false
  sendAnonymousUsage: false

log:
  level: INFO
  format: json

api:
  dashboard: true
  insecure: false # Dashboard uniquement via HTTPS

entryPoints:
  web:
    address: ":80"
    http:
      redirections:
        entryPoint:
          to: websecure
          scheme: https
          permanent: true
  websecure:
    address: ":443"

certificatesResolvers:
  acme-cloudflare:
    acme:
      email: romain@romain-ecarnot.com
      storage: /data/acme.json
      # Utiliser Let's Encrypt (production) :
      caServer: https://acme-v02.api.letsencrypt.org/directory
      # Décomment pour Let's Encrypt staging (tests) :
      # caServer: https://acme-staging-v02.api.letsencrypt.org/directory
    dnsChallenge:
      provider: cloudflare # ← provider Cloudflare
      resolvers:
        - "1.1.1.1:53" # DNS Cloudflare
        - "8.8.8.8:53" # DNS Google (fallback)
      delayBeforeCheck: 30 # Secondes avant vérification
```

## 4.3 — Docker Compose complet

Le fichier docker-compose.yml injecte CF\_DNS\_API\_TOKEN comme variable d'environnement dans le conteneur Traefik :

```
docker-compose.yml — Service Traefik

# docker-compose.yml
version: '3.9'

networks:
  traefik-public:
    external: true # docker network create traefik-public
  internal:
    internal: true

services:
  traefik:
    image: traefik:v3.3
    container_name: traefik
    restart: unless-stopped
    security_opt:
      - no-new-privileges:true
    ports:
      - "80:80"
      - "443:443"
    environment:
      # Injecté via export depuis SOPS avant docker compose up
      CF_DNS_API_TOKEN: ${CF_DNS_API_TOKEN}
      # Optionnel — si votre provider requiert l'email CF :
      CF_API_EMAIL: ${CF_API_EMAIL:-romain@romain-ecarnot.com}
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - ./traefik.yml:/traefik.yml:ro
      - ./config:/config:ro
      - ./data/acme.json:/data/acme.json
    networks:
      - traefik-public
    labels:
      - "traefik.enable=true"
      # Dashboard Traefik sécurisé :
      - "traefik.http.routers.traefik.rule=Host(`traefik.romain-ecarnot.com`)"
      - "traefik.http.routers.traefik.entrypoints=websecure"
      - "traefik.http.routers.traefik.tls.certresolver=acme-cloudflare"
      - "traefik.http.routers.traefik.service=api@internal"
      - "traefik.http.routers.traefik.middlewares=traefik-auth"
      # Basic auth (htpasswd) :
      - "traefik.http.middlewares.traefik-auth.basicauth.users=romain:$2y$$. . ."
```

## 4.4 — Certificats ACME : détail du DNS Challenge

Le DNS Challenge permet d'obtenir des certificats wildcard (\*.romain-ecarnot.com) et fonctionne sans exposer le port 80 au public. Voici comment Traefik interagit avec l'API Cloudflare :

| Étape | Action                                                     | API Cloudflare utilisée |
|-------|------------------------------------------------------------|-------------------------|
| 1     | Traefik demande un certificat à Let's Encrypt              | Aucune — contact ACME   |
| 2     | LE envoie un challenge DNS (_acme-challenge.domain)        | Aucune                  |
| 3     | Traefik crée l'enregistrement TXT via CF API               | Zone:DNS:Edit ← requis  |
| 4     | LE vérifie l'enregistrement DNS (attente delayBeforeCheck) | Zone:Zone:Read ← requis |
| 5     | LE délivre le certificat, Traefik supprime le TXT          | Zone:DNS:Edit ← requis  |

## 4.5 — Services applicatifs avec TLS automatique

Exemple d'un service web derrière Traefik avec certificat wildcard :

```
docker-compose.yml + scripts/start.sh

# Extrait docker-compose.yml – Ajout d'un service applicatif
mon-app:
  image: nginx:alpine
  container_name: mon-app
  restart: unless-stopped
  networks:
    - traefik-public
    - internal
  labels:
    - "traefik.enable=true"
    # Routing :
    - "traefik.http.routers.mon-app.rule=Host(`app.romain-ecarnot.com`)"
    - "traefik.http.routers.mon-app.entrypoints=websecure"
    # Certificat TLS via DNS Challenge :
    - "traefik.http.routers.mon-app.tls=true"
    - "traefik.http.routers.mon-app.tls.certresolver=acme-cloudflare"
    # Wildcard optionnel (*.romain-ecarnot.com) :
    - "traefik.http.routers.mon-app.tls.domains[0].main=romain-ecarnot.com"
    - "traefik.http.routers.mon-app.tls.domains[0].sans=*.romain-ecarnot.com"
    # Service :
    - "traefik.http.services.mon-app.loadbalancer.server.port=80"

# Script de démarrage complet : scripts/start.sh
#!/bin/bash
set -euo pipefail

# Déchiffrer et injecter le token
export CF_DNS_API_TOKEN=$(sops -d \
  --extract '["cloudflare"]["CF_DNS_API_TOKEN"]' secrets.enc.yaml)

# S'assurer que acme.json existe avec les bonnes permissions
mkdir -p data && touch data/acme.json && chmod 600 data/acme.json

# Créer le réseau si inexistant
docker network create traefik-public 2>/dev/null || true

# Démarrer Traefik
docker compose up -d
echo "✓ Traefik démarré – Dashboard: https://traefik.romain-ecarnot.com"
```

## 5

## Vérifications et erreurs courantes

Tests · Diagnostic · Solutions

## 5.1 — Tests de validation

```

terminal — Tests de validation

# 1. Vérifier que Traefik tourne correctement
docker compose ps traefik
docker compose logs traefik --tail=50

# 2. Vérifier que la variable CF_DNS_API_TOKEN est bien injectée
docker exec traefik env | grep CF_DNS_API_TOKEN
# Attendu : CF_DNS_API_TOKEN=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# 3. Tester la connectivité vers l'API Cloudflare depuis le conteneur
docker exec traefik wget -qO- https://api.cloudflare.com/client/v4/user/tokens/verify \
  --header "Authorization: Bearer $CF_DNS_API_TOKEN"

# 4. Vérifier les certificats ACME
cat data/acme.json | jq '.acme-cloudflare.Certificates[].domain'

# 5. Test TLS depuis l'extérieur
curl -vI https://app.romain-ecarnot.com 2>&1 | grep -E 'SSL|TLS|expire'
openssl s_client -connect app.romain-ecarnot.com:443 </dev/null 2>&1 | grep -E 'CN|expire'

```

## 5.2 — Erreurs fréquentes et solutions

**CF\_DNS\_API\_TOKEN not set ou vide**

Cause : La variable d'environnement n'est pas injectée dans le conteneur.

Solution : Vérifier que `export CF_DNS_API_TOKEN=...` est exécuté AVANT `docker compose up`. Contrôler avec : `docker exec traefik env | grep CF_DNS`**401 Unauthorized — Cloudflare API**

Cause : Le token est invalide, expiré, ou mal copié.

Solution : Retester via curl : `curl -H 'Authorization: Bearer TOKEN' https://api.cloudflare.com/client/v4/user/tokens/verify`**⚠ 403 Forbidden — permission insuffisante**Cause : Le token n'a pas les permissions `Zone:DNS>Edit` ou `Zone:Zone:Read`.Solution : Vérifier dans le dashboard CF que les deux permissions sont présentes. Attention : `Zone:DNS>Edit` ≠ `Zone:DNS:Read` — la permission doit être Edit.**⚠ DNS challenge timeout / delayBeforeCheck trop court**

Cause : Le DNS n'est pas encore propagé quand Let's Encrypt vérifie.

Solution : Augmenter `delayBeforeCheck` à 60 ou 90 secondes dans `traefik.yml`. Utiliser `1.1.1.1:53` comme resolver.

**⚠ acme.json : permission denied**

Cause : Le fichier acme.json n'a pas les bonnes permissions.

Solution : Exécuter : `chmod 600 data/acme.json` — Traefik refuse de démarrer si les permissions sont trop larges (sécurité).

**⚠ Too many certificates — Let's Encrypt rate limit**

Cause : Limite de 5 certificats par domaine par semaine dépassée.

Solution : Utiliser le serveur staging pour les tests : caServer: <https://acme-staging-v02.api.letsencrypt.org/directory>

**SOPS: no matching creation rules**

Cause : Le fichier .sops.yaml est absent ou mal configuré.

Solution : Vérifier que .sops.yaml est dans le répertoire courant. Vérifier que path\_regex correspond au nom du fichier.

**SOPS: failed to decrypt: age: no identities found**

Cause : La clé privée age n'est pas accessible.

Solution : Vérifier SOPS\_AGE\_KEY\_FILE ou que ~/.config/sops/age/keys.txt existe. Source ~/.zshrc si la variable d'env n'est pas chargée.

## 5.3 — Commandes de diagnostic rapide

```
terminal — Diagnostic

# Logs Traefik en temps réel (filtrés)
docker compose logs -f traefik | grep -E 'error|ACME|cloudflare|certificate'

# État des certificats ACME stockés
docker exec traefik cat /data/acme.json | python3 -m json.tool | head -60

# Vérifier les enregistrements DNS TXT cloudflare (pendant challenge)
dig TXT _acme-challenge.romain-ecarnot.com @1.1.1.1

# Tester la résolution SOPS complète
SOPS_AGE_KEY_FILE=~/.config/sops/age/keys.txt sops --decrypt secrets.enc.yaml

# Vérifier que Docker a bien la variable CF_DNS_API_TOKEN
docker inspect traefik | jq '[0].Config.Env[] | select(startswith("CF_"))'

# Redémarrer Traefik et forcer le renouvellement des certificats
docker compose restart traefik

# Si acme.json est corrompu, le supprimer et redémarrer :
rm data/acme.json && touch data/acme.json && chmod 600 data/acme.json
docker compose restart traefik
```

Ressources officielles : [doc.traefik.io](https://doc.traefik.io) — [ACME · developers.cloudflare.com](https://developers.cloudflare.com) — [API Tokens · getsops.io](https://getsops.io) — [Documentation SOPS · github.com/FiloSottile/age](https://github.com/FiloSottile/age)